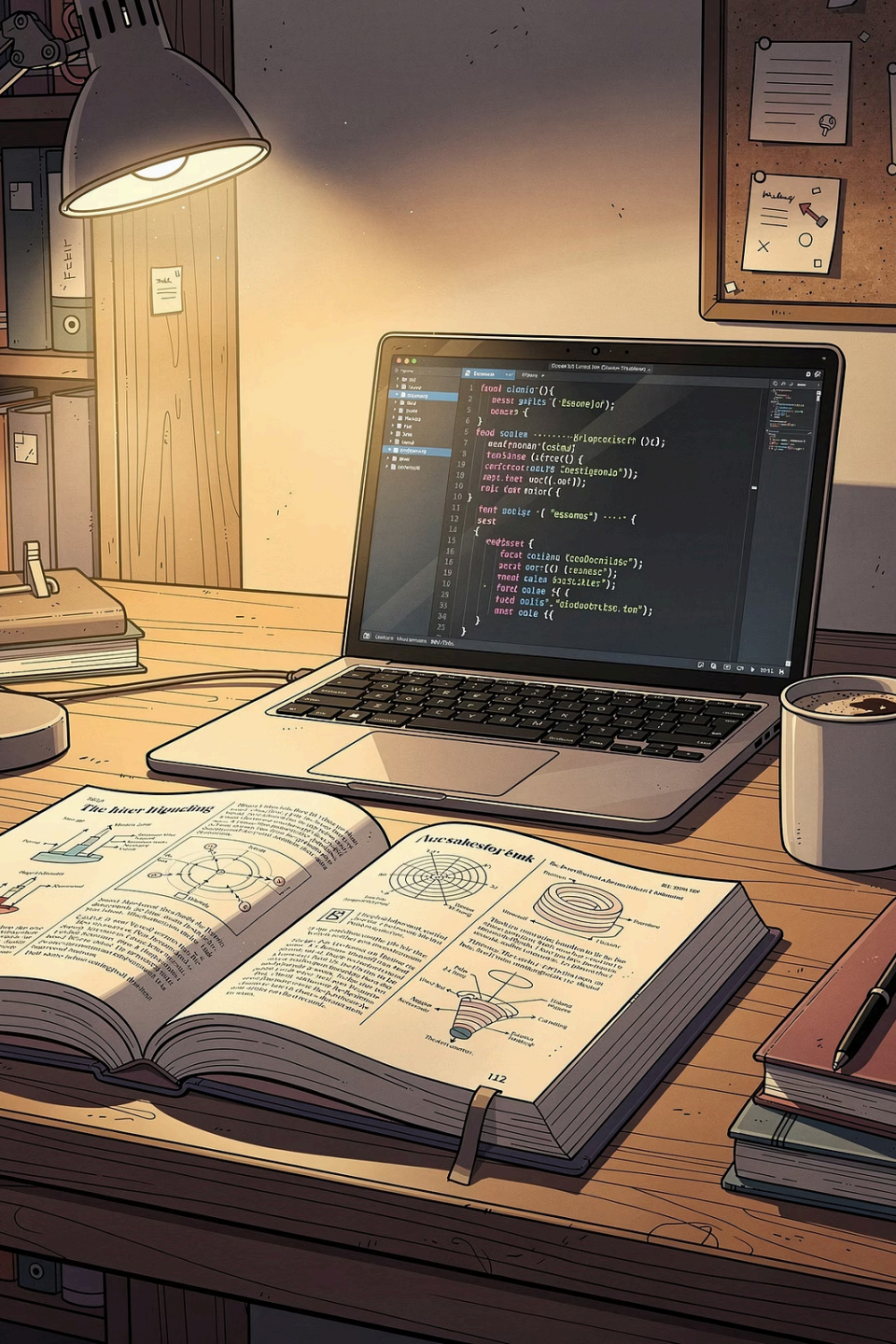




Chapter 8: Dictionaries and Sets

Course 040223101 · Computer Programming for Mathematics 1

Department of Mathematics, Faculty of Applied Science, King Mongkut's
University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul · Semester 1, Academic Year
2026

Learning Objectives

Upon completing this chapter, students will be able to:

O1

Create & Access Dictionaries

Build dictionaries and access data through key - value pairs.

O2

Use Dictionary Methods

Apply methods such as keys, values, items, and get.

O3

Create & Operate on Sets

Build sets and perform union, intersection, and difference operations.

O4

Choose the Right Structure

Select dictionaries or sets appropriately based on the nature of the problem.

O5

Apply to Counting & Grouping

Use dictionaries and sets to count frequencies and group data.
(CLO 4)

Chapter Overview: Why Dictionaries and Sets?

The Problem with Lists

Lists store data in order and are accessed by a numeric index. But sometimes we want to access data by **name** – for example, looking up a student's score by their name rather than by position number.

Two New Structures

This chapter introduces two powerful data structures:

- **Dictionary** – stores data as key - value pairs, accessed by key instead of numeric index.
- **Set** – stores unique, unordered members and supports mathematical set operations.

Both structures complement lists, giving us the right tool for the right problem.

8.1 What Is a Dictionary?

A Python dictionary stores data as **key : value** pairs. It is created using curly braces `{ }`. Values are accessed using their key – not a numeric index. Keys must be **unique** and **immutable** (e.g., strings or numbers).



Dictionary: Basic Syntax

File: **o8dict1.py**

```
student = {"name": "Somchai", "age": 19, "gpa": 3.5}

print(student["name"])
print(student["gpa"])

student["age"] = 20    # modify a value
student["major"] = "Math" # add a new pair
print(student)
```

Output:

```
Somchai
3.5
{'name': 'Somchai', 'age': 20, 'gpa': 3.5, 'major': 'Math'}
```

⚠ If you access a key that does not exist using `student["xyz"]`, Python raises a `KeyError`. Use the `get()` method instead – it returns `None` (or a specified default value) rather than throwing an error.

Key Properties of Dictionaries

Key-Value Pairs

Every entry is a pair: a unique key maps to a value. Keys act like labels for their associated data.

Mutable

Dictionaries can be modified after creation – values can be updated, new pairs added, and existing pairs removed.

Keys Must Be Immutable

Keys must be of an immutable type such as strings, numbers, or tuples. Lists cannot be used as keys.

No Duplicate Keys

Each key must be unique within a dictionary. Assigning a value to an existing key overwrites the previous value.

8.2 Dictionary Methods

Python provides several built-in methods for working with dictionaries efficiently.

Method	Meaning
<code>d.keys()</code>	Returns all keys in the dictionary
<code>d.values()</code>	Returns all values in the dictionary
<code>d.items()</code>	Returns all (key, value) pairs
<code>d.get(k, default)</code>	Returns value of key <code>k</code> ; returns <code>default</code> if key not found
<code>d.pop(k)</code>	Removes and returns the value of key <code>k</code>
<code>k in d</code>	Checks whether key <code>k</code> exists (returns <code>True/False</code>)

Iterating Over a Dictionary

We can loop over a dictionary using a `for` loop – either over keys alone or over key - value pairs using `.items()`.

File: **o8dict2.py**

```
scores = {"Ann": 85, "Bob": 90, "Cat": 78}

for name, score in scores.items():
    print(f"{name} scored {score} points")

print("Average:", sum(scores.values()) / len(scores))
```

Output:

```
Ann scored 85 points
Bob scored 90 points
Cat scored 78 points
Average: 84.33333333333333
```

 Using `.items()` unpacks each entry into two variables simultaneously – a clean and readable pattern for processing all key - value pairs.

Using `get()` vs. Direct Access

Direct Access – Risky

```
d = {"a": 1}
print(d["b"])
# KeyError: 'b'
```

Accessing a missing key with bracket notation raises a `KeyError` and crashes the program.

Using `get()` – Safe

```
d = {"a": 1}
print(d.get("b", 0))
# Output: 0
```

The `get()` method returns the default value (here `0`) when the key is not found – no error is raised. This is the recommended approach for safe dictionary access.



8.3 Applied Example: Frequency Counting

Dictionaries are ideal for counting the frequency of items in data. The pattern uses `get()` to safely increment a counter for each item encountered.

Frequency Counting: Code Example

File: **o8freq.py** – Count how many times each character appears in a string.

```
text = "mathematics"
freq = {}

for ch in text:
    freq[ch] = freq.get(ch, 0) + 1

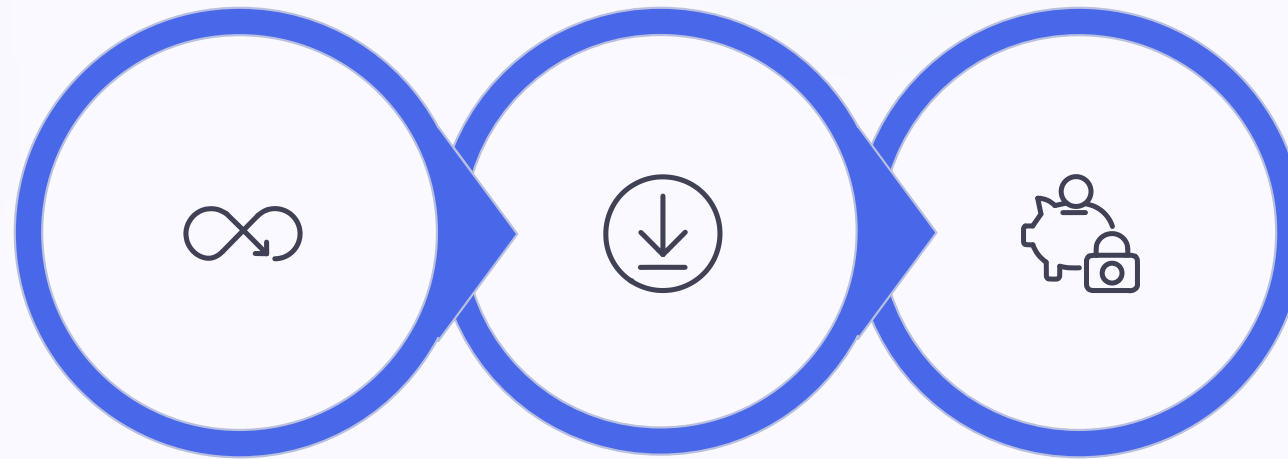
print(freq)
```

Output:

```
{'m': 2, 'a': 2, 't': 2, 'h': 1, 'e': 1, 'i': 1, 'c': 1, 's': 1}
```

 The pattern `freq.get(ch, 0) + 1` is the standard idiom for frequency counting: if the key exists, increment its count; if not, start from 0 and add 1.

How Frequency Counting Works



Iterate chars

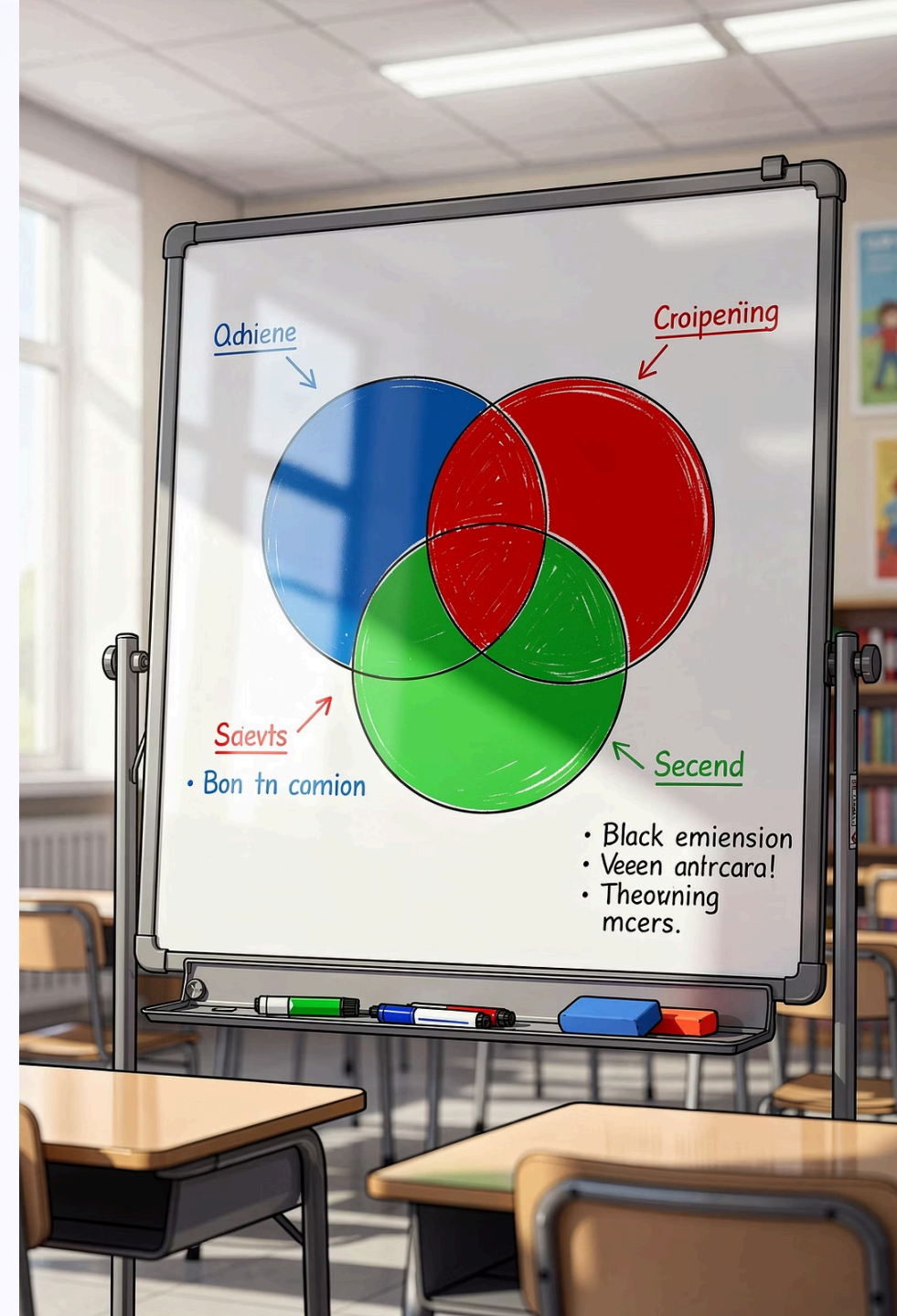
Get count

Store +1

This three-step loop pattern is one of the most common and powerful uses of dictionaries in Python. It generalizes to counting words, categories, or any discrete values in a dataset.

8.4 What Is a Set?

A **set** is a collection of members that are **unique** and **unordered**. Sets are created using curly braces `{ }` or the `set()` function. When duplicate values are inserted, the set retains only one copy – making sets ideal for eliminating duplicates.



Creating and Modifying Sets

File: **o8set1.py**

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)
print(unique)
print("Number of unique values:", len(unique))

unique.add(5)
unique.discard(1)
print(unique)
```

Output:

```
{1, 2, 3, 4}
Number of unique values: 4
{2, 3, 4, 5}
```

 Use `.add(x)` to insert an element and `.discard(x)` to remove one safely (no error if the element is not present). Use `.remove(x)` if you want an error when the element is missing.

Key Properties of Sets



No Duplicates

Every element in a set is unique. Adding a value that already exists has no effect on the set.



Unordered

Sets do not maintain insertion order. You cannot access elements by index as you would with a list.



Mathematical Operations

Sets support union, intersection, difference, and symmetric difference – mirroring mathematical set theory.



Deduplication Tool

Converting a list to a set with `set()` is the fastest way to remove all duplicate values.

8.5 Set Operations

Sets support the same mathematical operations as sets in mathematics. Each operation has both an operator shorthand and a method form.

Operation	Operator	Method	Meaning
Union	<code>A B</code>	<code>A.union(B)</code>	Members in A or B (or both)
Intersection	<code>A & B</code>	<code>A.intersection(B)</code>	Members in both A and B
Difference	<code>A - B</code>	<code>A.difference(B)</code>	Members in A but not in B
Symmetric Difference	<code>A ^ B</code>	<code>A.symmetric_difference(B)</code>	Members in A or B but not both

Set Operations: Code Example

File: **o8setop.py**

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7}
```

```
print("Union:", A | B)
```

```
print("Intersection:", A & B)
```

```
print("Difference A-B:", A - B)
```

```
print("Symmetric Difference:", A ^ B)
```

Output:

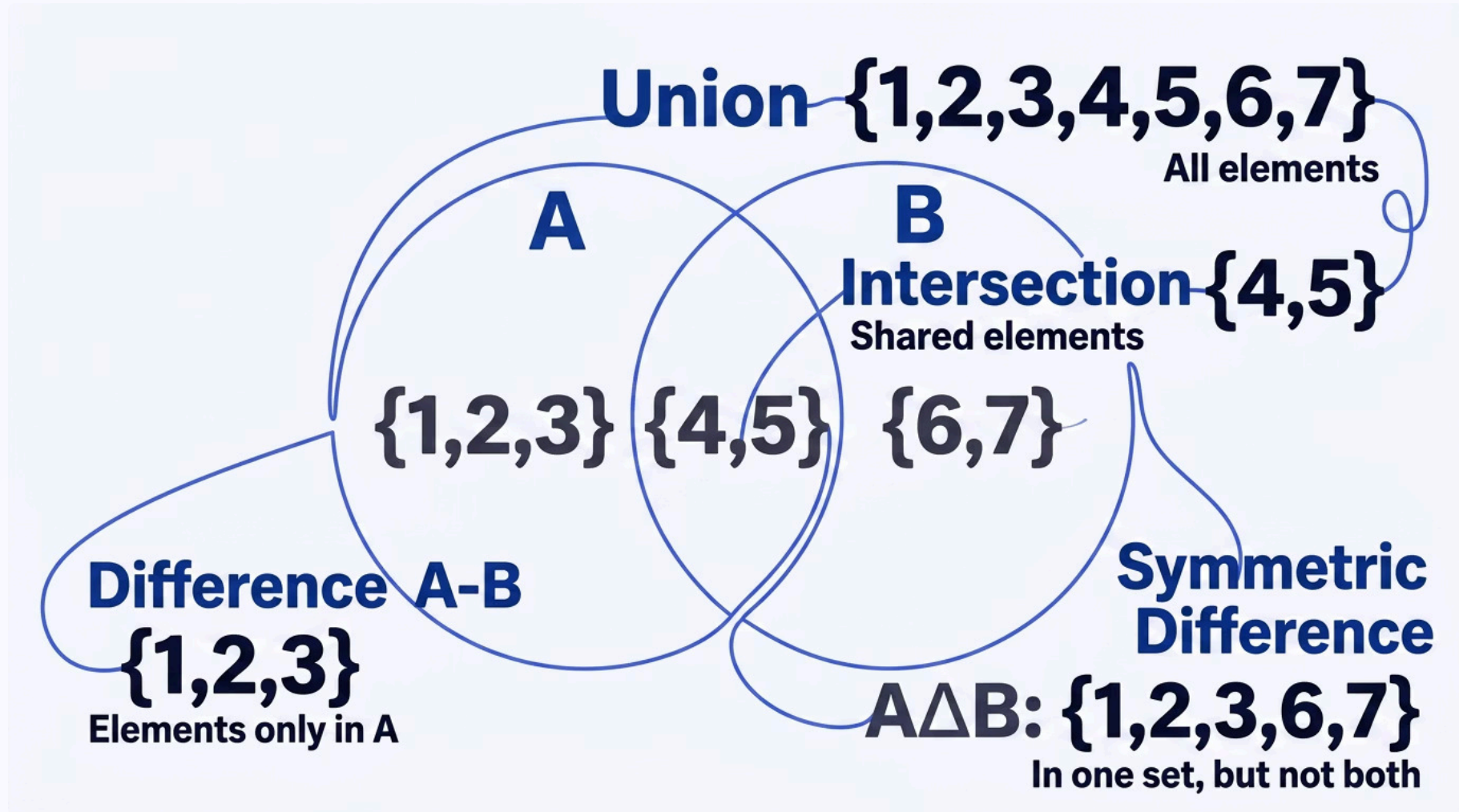
```
Union: {1, 2, 3, 4, 5, 6, 7}
```

```
Intersection: {4, 5}
```

```
Difference A-B: {1, 2, 3}
```

```
Symmetric Difference: {1, 2, 3, 6, 7}
```

Visualizing Set Operations



The Venn diagram illustrates how each set operation selects different regions of the two sets. Union covers everything; intersection covers only the overlap; difference covers one side only; symmetric difference covers both sides but excludes the overlap.

Set Operations: Step-by-Step Breakdown

1

Union $A \cup B$

Combines all members from both sets. Result: $\{1, 2, 3, 4, 5, 6, 7\}$. No duplicates are kept.

2

Intersection $A \cap B$

Keeps only members present in **both** sets. Result: $\{4, 5\}$.

3

Difference $A - B$

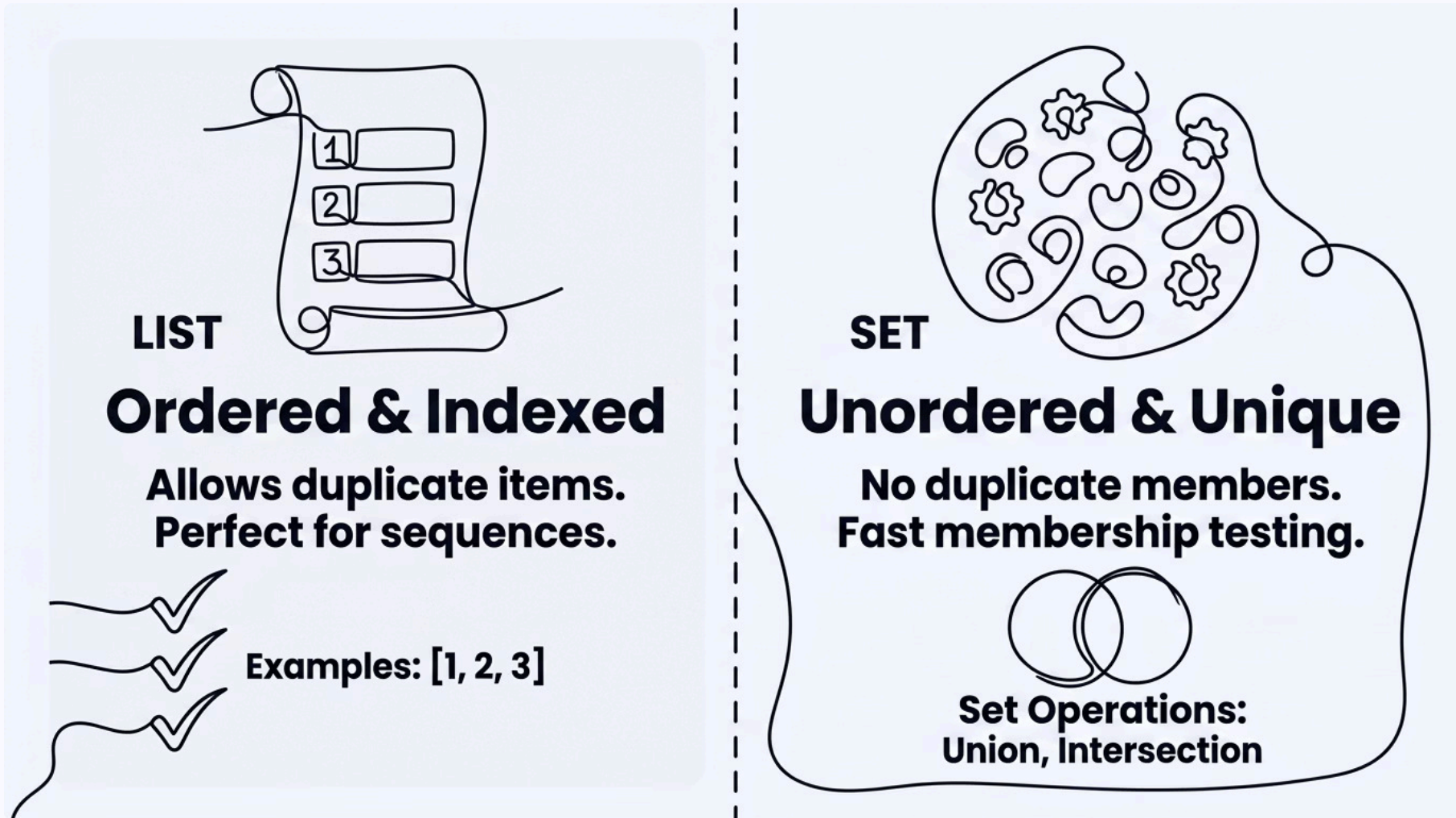
Keeps members in A that are **not** in B. Result: $\{1, 2, 3\}$. Note: $A - B \neq B - A$.

4

Symmetric Difference $A \oplus B$

Keeps members in A or B but **not both**. Result: $\{1, 2, 3, 6, 7\}$.
Equivalent to $(A \cup B) - (A \cap B)$.

Comparing Dictionaries and Sets



Choosing the right data structure depends on the problem. Use lists for ordered sequences, dictionaries for named lookups and frequency counting, and sets for uniqueness and mathematical set operations.



Chapter Summary: 8.6

This chapter covered two essential Python data structures that extend our toolkit beyond lists.

Summary: Dictionaries

Key-Value Storage

Dictionaries store data as key - value pairs and are accessed by key rather than numeric index – ideal for named lookups.

Essential Methods

The most important methods are `keys()`, `values()`, `items()`, and `get()`. Use `get()` for safe access without `KeyError`.

Best Use Cases

Dictionaries excel at data pairing (e.g., name → score) and frequency counting (e.g., character or word counts).

Summary: Sets

Unique, Unordered Members

Sets store only unique values with no defined order. Duplicate values are automatically discarded upon insertion.

Set Operations

Sets support union ($\cup$), intersection ($\cap$), difference ($-$), and symmetric difference (Δ), mirroring mathematical set theory.

Best Use Cases

Sets are ideal for removing duplicates from a list and for checking membership or computing overlaps between groups of data.

End-of-Chapter Exercises

1 Exercise 1 – Oldest Friend

Create a dictionary storing the names and ages of 3 friends.
Display the name of the person with the highest age.

3 Exercise 3 – Unique Values

Write a program that finds the number of unique values in the list `[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`.

2 Exercise 2 – Word Frequency

Write a program that counts the frequency of each word in a sentence entered by the user (split words by spaces).

4 Exercise 4 – Set Operations

Given `A = {2,4,6,8,10}` and `B = {3,6,9}`, display the results of union, intersection, and `A-B`.

Exercise Solutions

The following cards present the model solutions for Exercises 2, 3, and 4 from the end-of-chapter exercises.



Solution: Exercise 2 – Word Frequency

Count the frequency of each word in a user-entered sentence, splitting by spaces.

```
sentence = input("Enter a sentence: ")
words = sentence.split()
freq = {}

for w in words:
    freq[w] = freq.get(w, 0) + 1

print(freq)
```

Sample Output:

```
Enter a sentence: a b a c b a
{'a': 3, 'b': 2, 'c': 1}
```

- ✔ This solution reuses the frequency counting pattern from Section 8.3, applied to words instead of characters. The `split()` method divides the sentence into a list of words by whitespace.

Solution: Exercise 3 – Unique Values

Find the number of unique values in the list [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5].

```
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
print("Unique values:", set(nums))
print("Count:", len(set(nums)))
```

Output:

```
Unique values: {1, 2, 3, 4, 5, 6, 9}
Count: 7
```

- ✔ Converting a list to a set with `set()` instantly removes all duplicates. The original list has 11 elements, but only 7 are unique. Using `len(set(nums))` gives the count in a single expression.

Solution: Exercise 4 – Set Operations

Given $A = \{2, 4, 6, 8, 10\}$ and $B = \{3, 6, 9\}$, compute union, intersection, and $A-B$.

```
A = {2, 4, 6, 8, 10}
```

```
B = {3, 6, 9}
```

```
print(A | B) # Union
```

```
print(A & B) # Intersection
```

```
print(A - B) # Difference
```

Output:

```
{2, 3, 4, 6, 8, 9, 10}
```

```
{6}
```

```
{2, 4, 8, 10}
```

- ✔ Only the element 6 appears in both sets, so the intersection is {6}. The difference $A-B$ removes only 6 from A (since 3 and 9 are not in A), leaving {2, 4, 8, 10}.

Dictionary vs. List: When to Use Which

Use a List When...

- Order matters and you need index-based access
- Data is a simple sequence (e.g., a list of scores)
- You need to allow duplicate values
- You iterate through items in order

Use a Dictionary When...

- You need to look up data by a meaningful name (key)
- You are mapping one value to another (e.g., name → score)
- You are counting frequencies of items
- You need fast membership testing by key

Set vs. List: When to Use Which

Use a Set When...

- You need to store only unique values
- You want to remove duplicates from a list quickly
- You need to perform union, intersection, or difference
- Order does not matter

Use a List When...

- Order and position matter
- Duplicates are meaningful (e.g., recording repeated events)
- You need index-based access
- You need to sort or slice the data

The Frequency Counting Pattern in Depth

The frequency counting pattern is one of the most versatile dictionary applications. It generalizes to any discrete data.



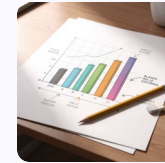
Character Frequency

Loop over each character in a string. Use `freq.get(ch, 0) + 1` to count occurrences. Applied in Section 8.3 to the word "mathematics".



Word Frequency

Split a sentence into words with `.split()`, then apply the same counting loop. Used in Exercise 2 solution to count words in user input.



General Item Frequency

The same pattern works for any iterable – counting grades, categories, survey responses, or any discrete values in a dataset.

Nested Dictionaries

Dictionary values can themselves be dictionaries, enabling structured, hierarchical data storage.

```
students = {  
    "Somchai": {"age": 19, "gpa": 3.5},  
    "Malee": {"age": 20, "gpa": 3.8}  
}  
  
print(students["Somchai"]["gpa"]) # Output: 3.5  
print(students["Malee"]["age"]) # Output: 20
```

- ❗ Nested dictionaries are useful for representing records with multiple fields – similar to a row in a database table. Access nested values by chaining bracket notation or `get()` calls.

Dictionary Comprehension

Just as list comprehensions create lists in one line, **dictionary comprehensions** create dictionaries concisely.

```
# Create a dictionary of squares: {1:1, 2:4, 3:9, 4:16, 5:25}
squares = {x: x**2 for x in range(1, 6)}
print(squares)
# Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
# Filter: only even keys
even_squares = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(even_squares)
# Output: {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

- ① Dictionary comprehensions follow the syntax `{key_expr: value_expr for item in iterable}`, optionally with a condition. They are concise and Pythonic for building dictionaries from computed data.

Set Comprehension

Sets can also be built using comprehension syntax – useful for generating unique values from a computation.

```
# Set of unique first letters from a list of words
words = ["apple", "banana", "avocado", "blueberry", "cherry"]
first_letters = {w[0] for w in words}
print(first_letters)
# Output: {'a', 'b', 'c'}

# Set of squares of odd numbers 1–9
odd_squares = {x**2 for x in range(1, 10) if x % 2 != 0}
print(odd_squares)
# Output: {1, 9, 25, 49, 81}
```

- ❗ Set comprehensions use curly braces like dictionary comprehensions but contain only a single expression (no colon). The result is always a set of unique values.

Practical Application: Grouping Data

Dictionaries can group items by category – a common data processing task.

```
students = [  
    ("Ann", "Math"), ("Bob", "CS"), ("Cat", "Math"),  
    ("Dan", "CS"), ("Eve", "Math")  
]  
  
groups = {}  
for name, major in students:  
    if major not in groups:  
        groups[major] = []  
    groups[major].append(name)  
  
print(groups)  
# Output: {'Math': ['Ann', 'Cat', 'Eve'], 'CS': ['Bob', 'Dan']}
```

- ✔ This grouping pattern – checking if a key exists, initializing a list if not, then appending – is a fundamental technique for organizing data by category using dictionaries.

Practical Application: Set for Membership Testing

Sets provide very fast membership testing – much faster than lists for large datasets.

```
enrolled = {"Ann", "Bob", "Cat", "Dan", "Eve"}
```

```
checked_in = {"Ann", "Cat", "Eve"}
```

```
# Who is enrolled but has NOT checked in?
```

```
absent = enrolled - checked_in
```

```
print("Absent:", absent)
```

```
# Output: Absent: {'Bob', 'Dan'}
```

```
# Is "Bob" enrolled?
```

```
print("Bob enrolled:", "Bob" in enrolled)
```

```
# Output: Bob enrolled: True
```

 The `in` operator on a set runs in $O(1)$ average time, compared to $O(n)$ for a list. For large collections, using a set for membership testing is significantly more efficient.

Common Mistakes to Avoid

KeyError on Missing Key

Accessing `d["missing_key"]` raises a `KeyError`. Always use `d.get("missing_key", default)` or check with `"key" in d` first.

Confusing `{ }` for Empty Set

`s = { }` creates an **empty dictionary**, not an empty set. To create an empty set, use `s = set()`.

Assuming Set Order

Sets are unordered. Do not rely on the printed order of set elements – it may differ between runs or Python versions.

Using Mutable Keys

Lists and other mutable objects cannot be used as dictionary keys. Use strings, numbers, or tuples as keys instead.

Quick Reference: Dictionary Methods

`keys()`

Returns a view of all keys.
Iterate with `for k in d.keys()`.

`values()`

Returns a view of all values.
Use with `sum()`, `max()`, etc.

`items()`

Returns (key, value) pairs.
Unpack in a `for` loop.

`get(k, v)`

Safe access – returns `v` if
key `k` is missing.

`pop(k)`

Removes key `k` and returns its value.

`k in d`

Returns `True` if key `k` exists in the dictionary.

Quick Reference: Set Operations

$A \mid B$

Union – all members from A or B.

$A \& B$

Intersection – members in both A and B.

$A - B$

Difference – members in A but not B.

$A \wedge B$

Symmetric Difference – in A or B but not both.

`add(x)`

Adds element `x` to the set.

`discard(x)`

Removes `x` safely (no error if absent).

Chapter 8 Complete

Dictionaries and Sets – two powerful structures that extend Python's data handling capabilities beyond ordered lists.

Dictionary

Key - value pairs · Named access · Frequency counting · Methods: `keys`, `values`, `items`, `get`

Set

Unique members · Unordered · Deduplication · Operations: union, intersection, difference, symmetric difference

Course 040223101 · Computer Programming for Mathematics 1 · CLO 4

